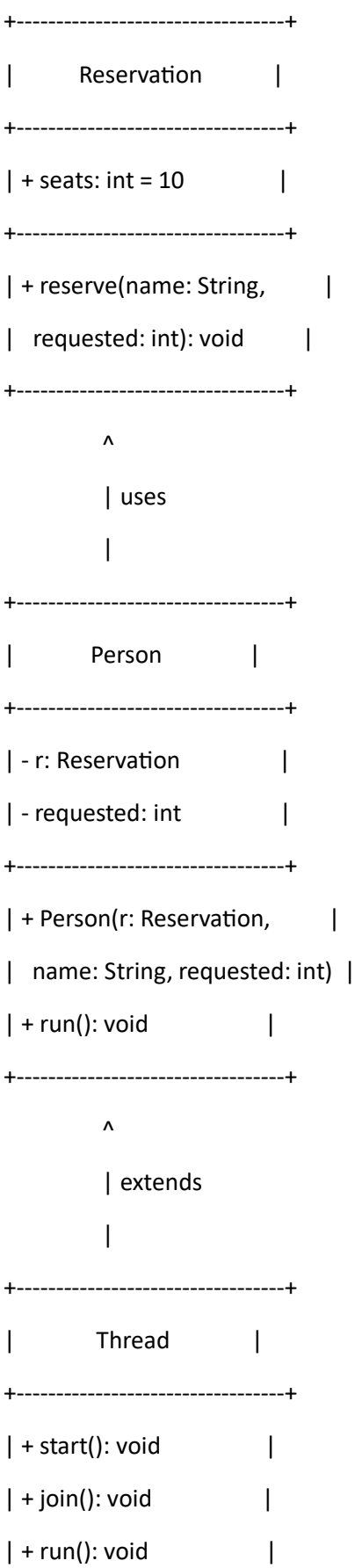
Question:

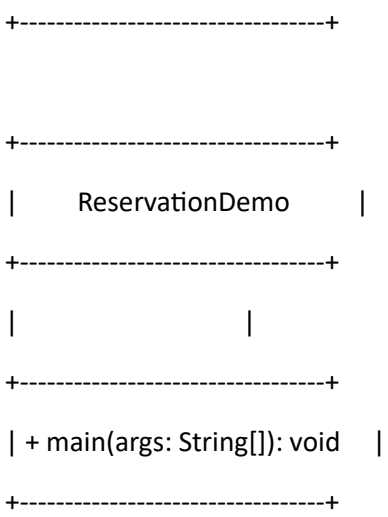
Implement a Reservation system which allows the persons to book seats. Define reserve method initially with 100 seats. Now create one or more person threads to book seats. At any time it should allow only one person thread to access the reserve method.

Algorithm:

1. Start.
2. Create a Reservation class with 10 seats.
3. Create a synchronized reserve() method
   to book seats safely.
4. Create a Person class extending Thread.
5. Override the run() method to call
   the reserve() method.
6. Create a Reservation object.
7. Create three threads:
   Person-1 requests 5 seats.
   Person-2 requests 2 seats.
   Person-3 requests 4 seats.
8. Start Person-1 and wait using join().
9. Start Person-2 and wait using join().
10. Start Person-3.
11. Check seat availability:
    If available:
      Reserve the requested seats.
      Update the remaining seats.
    Otherwise:
      Display seats not available.
12. Display the reservation details.
13. Stop.

UML Class Diagram:


Reservation
+ seats: int = 10
+ reserve(name: String,
requested: int): void
uses
Person
- r: Reservation
- requested: int
+ Person(r: Reservation,
name: String, requested: int)
+ run(): void
extends
Thread
+ start(): void
+ join(): void
+ run(): void



Code:

```
package javacore;

// Reservation class
class Reservation {
   int seats = 10;

   // Only one thread can access this method at a time
   synchronized void reserve(String name, int requested) {
      System.out.println(name + " entered.");

      System.out.println("Available seats: " + seats
            + " Requested seats: " + requested);

      if (seats >= requested) {
         System.out.println("Seat Available. Reserve now :-)");
         seats = seats - requested;
         System.out.println(requested + " seats reserved.");
      } else {
         System.out.println("Requested seats not available :-)");
```

```
        }

        System.out.println(name + " leaving.");
        System.out.println("-----------------------------------------");
    }
}

// Person thread class
class Person extends Thread {
    Reservation r;
    int requested;

    Person(Reservation r, String name, int requested) {
        super(name);
        this.r = r;
        this.requested = requested;
    }

    public void run() {
        r.reserve(getName(), requested);
    }
}

// Main class
public class ReservationDemo {
    public static void main(String[] args) {
        Reservation r = new Reservation();

        // Create three person threads
        Person p1 = new Person(r, "Person-1", 5);
        Person p2 = new Person(r, "Person-2", 2);
```

```
        Person p3 = new Person(r, "Person-3", 4);

        // Start the threads one by one
        p1.start();
        try {
            p1.join();
            p2.start();
            p2.join();
            p3.start();
        } catch (InterruptedException e) {
            System.out.println(e);
        }
    }
}
```

Output:

```
Person-1 entered.
Available seats: 10 Requested seats: 5
Seat Available. Reserve now :-)
5 seats reserved.
Person-1 leaving.
-----------------------------------------------
Person-2 entered.
Available seats: 5 Requested seats: 2
Seat Available. Reserve now :-)
2 seats reserved.
Person-2 leaving.
-----------------------------------------------
Person-3 entered.
Available seats: 3 Requested seats: 4
Requested seats not available :-)
```

Person-3 leaving.
